

Documento de Referencia: Seguimiento del Proyecto - Scrum

Proyecto:Aplicación: “No Te Cortes”

Sector: Servicios de Estética e Imagen Personal (CNAE 9602)

#21 Sprint 1: Estabilización de Infraestructura y Estructura NoSQL

- **Fecha de inicio:** 04/05/2026 | Fecha de fin: 10/05/2026
- **Estado:** Completado

1. Resumen de Actividades (Logros)

- **1:** Sincronización exitosa entre Android Studio y Firebase mediante google-services.json.
- **2:** Estructuración de la base de datos NoSQL en Firestore (colecciones: users, citas, services y promotions).
- **3:** Despliegue de Security Rules con lógica isAdmin() y control de acceso por UID.

2. Seguimiento de Tareas Técnicas

ID	Tarea	Estado	Observaciones Técnicas
#24 T1.1	Mapeo de Documentos NoSQL	100%	Clases Java mapeadas para persistencia en Firestore.
#25 T1.2	Integración de Servicios Backend	100%	Firebase Auth y SDK de Firestore operativos. Archivo google-services.json validado.
#26 T1.3	Definición del Esquema de Seguridad	100%	Reglas de seguridad probadas contra escrituras no autorizadas.
#27 T1.4	Protocolo de Trazabilidad	100%	Repositorio (ProyectoDAM) actualizado con commits atómicos.

3. Gestión de Incidencias Detectadas

#24 Incidencia 01: Error de Tipado en Documentos NoSQL

- **Tarea relacionada: T1.1** (Mapeo de Documentos NoSQL)
- **Descripción:** Se detectó que el campo precio en los documentos de la colección services estaba definido como String.
- **Problemas generados:** Errores de ejecución al intentar realizar cálculos aritméticos en la interfaz de usuario.
- **Cambios para futuros Sprints:** Se establece como norma obligatoria el uso de Double para valores monetarios y la revisión de tipos en el esquema NoSQL antes de programar la lógica de negocio.

#24 Incidencia 02: Fallo en el Control de Entrada (UX)

- **Tarea relacionada: T1.1** (Mapeo de Documentos NoSQL)
- **Descripción:** Los campos de precio y duración no automatizaban el teclado numérico en el dispositivo.
- **Problemas generados:** Riesgo alto de introducir caracteres alfanuméricos que provocan errores de tipo NumberFormatException y cierre inesperado de la App.
- **Cambios para futuros Sprints:** Revisión sistemática del atributo inputType en todos los ficheros XML. Se usarán exclusivamente numberDecimal y number para evitar entradas inválidas.

#26 Incidencia 03: Vulnerabilidad en Cuentas Generadas por Admin

- **Tarea relacionada: T1.3** (Definición del Esquema de Seguridad)
- **Descripción:** Las cuentas creadas manualmente por el administrador no obligaban a una actualización de credenciales.
- **Problemas generados:** Riesgo de seguridad por el uso de contraseñas temporales conocidas por terceros.
- **Cambios para futuros sprints:** Implementación del flag forcePasswordChange. Se incluirá en la lógica de todos los Sprints de acceso (Login) la redirección obligatoria a cambio de clave si el valor es true.

#25 Incidencia 04: Validación de Credenciales y Vinculación de API Backend

- **Tarea relacionada: T1.2** (Integración de Servicios Backend).
- **Descripción:** Necesidad de vincular de forma unívoca el paquete de la aplicación com.example.notecortes con el proyecto de Firebase para habilitar los servicios de Google.
- **Problemas generados:** Si la API Key o el SHA-1 no coinciden exactamente, los servicios de Auth y Firestore devuelven un error de "App not authorized", bloqueando cualquier intento de login o registro durante las pruebas iniciales.

- **Cambios para futuros sprints:** Se ha documentado la API Key en el registro de configuración. Para futuros Sprints que requieran nuevos servicios (como Storage o Functions), se deberá verificar siempre que el archivo google-services.json esté actualizado con el App ID correcto antes de subir cambios al repositorio.

#26 Incidencia 05: Blindaje de Integridad en Escrituras NoSQL

- **Tarea relacionada: T1.3** (Definición del Esquema de Seguridad).
- **Descripción:** Implementación de reglas de validación en el lado del servidor para evitar la manipulación de datos entre usuarios y proteger el catálogo.
- **Problemas generados:** Sin esta validación, cualquier usuario autenticado podría, mediante herramientas externas, modificar citas de otros clientes o alterar los precios de los servicios y promociones, comprometiendo la viabilidad del negocio.
- **Cambios para futuros sprints:** Se ha establecido el estándar de seguridad basado en el UID del propietario (`request.resource.data.userId == request.auth.uid`) para la colección Citas. Para futuros Sprints (como el desarrollo del panel Admin), se aplicará la restricción de rol estricta en las colecciones de Services y Promotions, garantizando que solo el rol de Administrador pueda realizar escrituras.

#28 Sprint 2: Seguridad y Flujo de Registro

- **Fecha de inicio:** 11/05/2026 | **Fecha de fin:** 17/05/2026
- **Estado:** Completado

1. Resumen de Actividades (Logros)

- **1:** Implementación del flujo de alta para el rol "Cliente" con validación en tiempo real.
- **2:** Activación del sistema de recuperación de cuentas mediante envío de email desde Firebase Auth.
- **3:** Despliegue de lógica de control para el campo sensible isAdmin, impidiendo la auto-asignación de privilegios.

2. Seguimiento de Tareas Técnicas

ID	Tarea	Estado	Observaciones Técnicas
#29 T2.1	Registro y Persistencia	100%	Solución de inconsistencias entre Auth y Firestore.
#30 T2.2	Validación de Formatos	100%	Integración de sendPasswordResetEmail funcional.
#31	Recuperación	100%	Control de tipos mediante RegEx (Email y

ID	Tarea	Estado	Observaciones Técnicas
T2.3	Password		Password).
#32 T2.4	Jerarquía de Roles	100%	Bloqueo de edición del campo isAdmin desde el cliente.

3. Gestión de Incidencias Detectadas

#29 Incidencia 06: Inconsistencia de Datos (Usuarios Huérfanos)

- **Tarea relacionada:** T2.1 (Registro y Persistencia)
- **Descripción:** Se detectó que el sistema completaba el registro en **Firestore Auth**, pero la escritura del documento de perfil en **Firestore** fallaba o se cancelaba por micro-cortes de red o latencia.
- **Problemas generados:** Usuarios que pueden hacer login pero no tienen perfil asociado, provocando errores de puntero nulo (NullPointerException) al intentar recuperar el String del nombre o el rol.
- **Cambios para futuros sprints:** Se ha sustituido la lógica lineal por el uso de **addOnCompleteListener** y **addOnFailureListener**.
- **Estrategia futura:** Para el motor de citas del Sprint 4, se implementará un sistema de **"Escritura Atómica"** o **Transacciones**, asegurando que el proceso solo se dé por válido si tanto la reserva como la actualización del calendario se confirman simultáneamente.

#30 Incidencia 07: Superposición de Mensajes de Error (UX)

- **Tarea relacionada:** T2.2 (Validación de Formatos)
- **Descripción:** Los mensajes de error de intentos fallidos anteriores permanecían visibles al iniciar un nuevo intento.
- **Problemas generados:** Confusión en el usuario, que visualizaba errores antiguos (ej: "Email no válido") tras haber corregido el campo, empeorando la experiencia de uso.
- **Cambios para futuros sprints:** Se añade una función de limpieza visual (**clearErrors()**) obligatoria al inicio de cada proceso. Este estándar se exportará a todos los formularios de la App.

#32 Incidencia 08: Error de Acceso en Alta de Usuario (Permission Denied)

- **Tarea relacionada:** T2.4 (Gestión de Privilegios).
- **Descripción:** Las reglas bloqueaban la creación del primer documento del usuario porque todavía no estaba "autenticado" ante la base de datos.

- **Problemas generados:** El registro de nuevos clientes era imposible, devolviendo errores de seguridad de Firebase.
- **Cambios para futuros sprints:** Ajuste de reglas para permitir create si el request.auth != null. Se revisarán las reglas de la colección citas para asegurar que no ocurra lo mismo al reservar la primera vez.

#32 Incidencia 09: Riesgo de Elevación de Privilegios

- **Tarea relacionada: T2.4** (Jerarquía de Roles)
- **Descripción:** El campo isAdmin era técnicamente accesible desde el código de la App en el registro inicial.
- **Problemas generados:** Un usuario malintencionado podría modificar el código para registrarse como administrador directamente.
- **Cambios para futuros sprints:** Blindaje absoluto en Firestore Rules donde la escritura en el campo isAdmin queda prohibida para cualquier UID que no tenga ya el rol de administrador validado en el servidor.

#31 Incidencia 10: Latencia en la recepción de Emails de Recuperación

- **Tarea relacionada: T2.3** (Ciclo de Vida de Sesión y Recuperación de Cuentas),
- **Descripción:** Durante las pruebas de estrés, el correo de restablecimiento de contraseña de Firebase tardaba más de 5 minutos en llegar o se clasificaba como Spam.
- **Problemas generados:** Frustración en el usuario y riesgo de abandono de la aplicación al pensar que el sistema no funcionaba.
- **Cambios para futuros sprints:** Configurar en la consola de Firebase una plantilla de correo más limpia y profesional.

#30 Incidencia 11: Simplificación Temporal de Validaciones

- **Tarea relacionada: T2.2** (Validación de Formatos)
- **Descripción:** Para no demorar la operativa crítica del registro y la conexión con el Backend, se ha implementado una validación básica de longitud mínima (>6\$ caracteres) en lugar de expresiones regulares (Regex) complejas.
- **Problemas generados:** Riesgo de que el usuario introduzca correos con formatos erróneos o contraseñas excesivamente simples en esta fase de desarrollo.
- **Cambios para futuros sprints:** Se programa para el Sprint 5 (QA y Pulido) la implementación de una clase ValidatorHelper con Regex para blindar la seguridad de los campos, una vez que el motor de citas y el CRUD de servicios sean totalmente funcionales.

Mejora de Diseño NoSQL: Indexación de Búsqueda por Email

- **Descripción:** Al intentar recuperar un perfil, Firestore devolvía errores si no existía un índice para buscar usuarios por su campo email.
- **Solución:** Se crearon índices compuestos en la consola de Firebase.
- **Cambios para futuros sprints:** Se establece como protocolo revisar la pestaña de "Índices" en Firebase cada vez que se cree una nueva colección (como Citas) que requiera filtros complejos (ej: buscar citas por fecha y peluquero a la vez).

#33 Sprint 3: Lógica de Disponibilidad y Ciclo de Vida

- **Periodo:** 18/05/2026 – 24/05/2026
- **Estado:** Completado

1. Resumen de Actividades (Logros)

- **Gestión de Agenda:** Implementación de restricciones en el DatePicker para bloquear fechas pasadas y domingos (días no laborables).
- **Robustez en Escritura:** Uso de Firebase Transactions para asegurar que dos clientes no puedan reservar la misma hora simultáneamente.
- **Navegación Avanzada:** Control total del flujo mediante onBackPressedCallback, garantizando que el usuario no regrese al Splash Screen una vez autenticado.
- **Optimización NoSQL:** Configuración de índices collectionGroup para permitir consultas transversales del historial de citas, mejorando el rendimiento de la App.

2. Seguimiento de Tareas Técnicas

ID	Tarea	Estado	Observaciones Técnicas
#34 T3.1	Enrutador Técnico de Roles	100%	Consultas síncronas a Firestore e Intents condicionales por perfil.
#35 T3.2	Control de Navegación (Anti-Back)	100%	Uso de finish() y la bandera FLAG_ACTIVITY_CLEAR_TASK en la RAM.
#36 T3.3	Dashboard Administrativo Base	100%	Estructuración visual de los accesos rápidos para la gerencia.
#37 T3.4	Localización de Excepciones	100%	Intercepción de errores de Firebase
#38 T3.5	Rediseño de Cambio de Clave	100%	Maquetación con Material Design 3 y toggle de visibilidad

ID	Tarea	Estado	Observaciones Técnicas
			de contraseña.
#39 T3.6	Sincronización de Ciclo de Vida Splash	100%	Corrección de fugas de memoria cancelando los callbacks del Handler en onDestroy().

3. Gestión de Incidencias y Mejoras

#34 Incidencia 12: Solapamiento de Citas

- **Tarea relacionada: T3.1** (Enrutador Técnico de Roles)
- **Descripción:** Se detectó que, si dos usuarios pulsaban "Reservar" al mismo tiempo, el sistema podía crear dos citas en el mismo hueco horario.
- **Problemas generados:** Conflictos en la agenda del administrador y mala experiencia de usuario al recibir confirmaciones duplicadas.
- **Cambios para futuros sprints:** Se sustituyó el método .set() básico por **Firestore Transactions**, lo que obliga al servidor a verificar la disponibilidad justo antes de escribir. Esta lógica se aplicará en el Sprint 4 para la edición de citas.

#35 Incidencia 13: Navegación Circular (Botón Atrás)

- **Tarea relacionada: T3.2** (Control de Navegación)
- **Descripción:** Al pulsar el botón físico "Atrás" desde el Home, la App regresaba al Login en lugar de cerrarse, a pesar de tener la sesión activa.
- **Problemas generados:** Inconsistencia en el flujo de seguridad y confusión en el usuario.
- **Cambios para futuros sprints:** Se implementó onBackPressedCallback con un sistema de cierre limpio (finishAffinity()). Se ha estandarizado este comportamiento para todas las actividades principales del proyecto.

#37 Incidencia 14: Mensajes de Error Ininteligibles

- **Tarea relacionada: T3.4** (Localización)
- **Descripción:** Los errores de red o credenciales aparecían en inglés técnico (ej: auth/network-request-failed).
- **Problemas generados:** El usuario final no comprendía por qué fallaba el acceso, restando profesionalidad a la aplicación.
- **Cambios para futuros sprints:** Creación de una clase ErrorManager que intercepta los códigos de Firebase y devuelve strings amigables desde strings.xml. En el Sprint 5 se añadirán iconos visuales a estos mensajes para mejorar la accesibilidad.
-

#34 Incidencia 15: Latencia en la Carga del Historial

- **Tarea relacionada:** T3.1 (Enrutador Técnico de Roles)
- **Descripción:** La consulta simple de citas era lenta a medida que crecía la base de datos.
- **Problemas generados:** "Lagueo" visual en la pestaña de historial del cliente.
- **Cambios para futuros sprints:** Se han habilitado **Índices de Grupo de Colecciones** en la consola de Firebase. Se monitorizará el consumo de lecturas para optimizar la paginación de datos en el próximo Sprint.

#40 Sprint 4: Motor de Reservas y Gestión Administrativa

- **Periodo:** 25/05/2026 – 31/05/2026
- **Estado:** Completado / Incremento de Software Validado

1. Resumen de Actividades (Logros)

- **1 (Motor de Reservas):** Persistencia bajo UID de cliente de forma única y flujo de reserva blindado.
- **2 (Gestión de Catálogo):** Alta, baja y modificación de servicios y promociones operativa al 100%.
- **3 (Control de Citas):** Validación del cambio de estados (Pendiente, Realizada, No Realizada) en vista de empleado.
- **4 (Optimización):** Consulta cronológica indexada funcional mediante OrderBy.

2. Seguimiento de Tareas Técnicas

ID	Tarea	Estado	Observaciones Técnicas
#41 T4.1	Panel CRUD Administrativo Reactivo	100%	Integración de SnapshotListener para sincronización en tiempo real.
#43 T4.2	Motor de Reservas y Validación	100%	Algoritmo de Minutos Totales y control de zona horaria Europe/Madrid.
#45 T4.3	Optimización por Desnormalización	100%	Reducción de lecturas incrustando datos en el documento transaccional.
#46 T4.4	Lógica de Estados de Agenda	100%	Máquina de estados (Pendiente, Realizada, Anulada) y alertas visuales.
#47	Administración de	100%	Creación de índices

ID	Tarea	Estado	Observaciones Técnicas
T4.5	Historial Indexado		compuestos para consultas por userId y fecha.

3. Gestión de Incidencias Detectadas

#41 Incidencia 16: Persistencia de hilos huérfanos y duplicidad de Activities

- **Tarea relacionada:** T4.1 (Panel CRUD Administrativo Reactivo)
- **Descripción:** Se detectó que el temporizador (Handler) del Splash Screen (MainActivity) continuaba su ejecución en segundo plano tras realizar el Login. Al agotarse el tiempo de espera (5 segundos), el hilo disparaba una nueva navegación hacia la HomeActivity.
- **Problemas generados:** Duplicidad de la interfaz de usuario (se abrían dos Homes una encima de otra). Al pulsar el botón "Atrás", el usuario se encontraba con una pantalla "fantasma" de la sesión anterior, rompiendo el flujo de navegación y consumiendo memoria innecesaria.
- **Cambios para futuros sprints:** Se ha implementado la cancelación obligatoria de todos los callbacks del Handler en el método onDestroy() de la actividad. Además, se ha centralizado el flujo de decisión en el LoginActivity para asegurar que el sistema solo tome una decisión de ruta por cada inicio de sesión.

#47 Incidencia 17: Sincronización de Privilegios y Roles (Admin)

- **Tarea Relacionada:** T4.5 (Gestión de roles)
- **Descripción:** El cambio de rol (Cliente a Empleado) no refrescaba los permisos en la sesión activa hasta reiniciar la App.
- **Problemas Generados:** Inconsistencia de permisos; el nuevo empleado no veía su agenda tras ser promocionado.
- **Solución:** Implementación de un **SnapshotListener** para detectar cambios en el campo role al instante y forzar la redirección automática de la interfaz.

#46 Incidencia 18: Lógica de cancelación de citas

- **Tarea Relacionada:** T4.4 (Lógica de Estados y Gestión Manual de Agenda).
- **Descripción:** El botón "Cancelar" ejecutaba la acción directamente sin solicitar confirmación previa del usuario.
- **Problemas Generados:** Cancelaciones accidentales e inconsistencia en la agenda del centro de estética.
- **Solución:** Uso obligatorio de **AlertDialog** de confirmación personalizado en color Rojo corporativo (**#D32F2F**) para acciones destructivas.

#47 Incidencia 19: Visualización ineficiente del historial

- **Tarea Relacionada: T4.5:** Módulo de historial.
- **Descripción:** Los datos se recuperaban en orden ascendente (las citas más antiguas aparecían primero).
- **Problemas Generados:** Mala usabilidad; el usuario debía realizar un scroll excesivo para consultar su última sesión o reserva.
- **Solución:** Aplicación del estándar técnico de consultas indexadas con **OrderBy("fecha", DESCENDING)** para toda la aplicación.

#41 Incidencia 20: Feedback ausente en edición de perfil

- **Tarea Relacionada: T4.1** (Panel CRUD Administrativo Reactivo)
- **Descripción:** La aplicación no emitía ningún aviso o señal tras guardar los cambios en los datos del perfil.
- **Problemas Generados:** Incertidumbre en el usuario y duplicidad de intentos de guardado (escrituras innecesarias en Firestore).
- **Solución:** Integración sistemática de **Snackbar** con acción de confirmación ("OK") para validar la persistencia de datos.

#47 Incidencia 21: Preservación de la Integridad Referencial (Baja Lógica)

- **Tarea Relacionada: T4.5** (Gestión de Perfiles e Historial) .
- **Descripción:** Se detectó que, si se eliminaba físicamente un documento de usuario de Firestore, las citas históricas asociadas perdían su referencia y causaban errores de carga.
- **Solución:** Implementación de un sistema de **"Baja Lógica"**. En lugar de borrar el documento, se modifica el campo activo: false. Esto permite deshabilitar el acceso al usuario sin romper el historial de citas del centro de estética.

#43 Incidencia 21.1: Desfase Horario en Validación de Citas

- **Tarea Relacionada: T4.2** (Motor de Reservas y Validación).
- **Descripción:** Se detectó un retardo de hasta 120 minutos en la validación de horas, permitiendo reservar citas que ya habían ocurrido según la hora local.
- **Solución:** Implementación de la zona horaria estática **Europe/Madrid** mediante la clase **TimeZone** y un algoritmo de cálculo basado en "Minutos Totales Diarios". Esto garantiza que el bloqueo sea exacto independientemente de la configuración del dispositivo del cliente.

#43 Incidencia 21.2: Error de Selección en Servicios de Larga Duración

- **Tarea Relacionada: T4.2** (Motor de Reservas y Validación).
- **Descripción:** El sistema solo validaba si el primer slot estaba libre, pero no comprobaba si los slots sucesivos (para servicios de 45 o 60 min) estaban disponibles.

- **Solución:** Programación de una **"Validación en Cascada"** que verifica la disponibilidad y caducidad de todos los bloques de tiempo necesarios según la duración del servicio seleccionado.

3. Configuración Técnica de Índices

- **Problema:** Excepción de ejecución al combinar filtros de userId con ordenación por fecha.
- **Acción Realizada:** Habilitación de **Índices Compuestos** manuales en la Consola de Firebase:
 - **Colección citas:** Campo userId (Ascendente) + fecha (Descendente).
 - **Colección citas:** Campo estado (Ascendente) + fecha (Descendente).
- **Resultado:** Garantía de carga instantánea y persistencia íntegra de la agenda sin errores de ejecución.

Estado Final: Incremento de software entregado y validado. Listo para la fase final de auditoría y cierre del proyecto (Sprint 5).

#48 Sprint 5: Auditoría Final

- **Periodo:** 01/06/2026 – 07/06/2026
- **Estado:** Completado

1. Resumen de Actividades (Logros)

- **Consistencia Visual:** Refactorización integral de interfaces a ConstraintLayout y unificación gráfica mediante el patrón **Header-Card** (Azul Corporativo #0D47A1) en los módulos de acceso.
- **Accesibilidad y Adaptabilidad:** Inclusión de ScrollView en formularios de entrada para garantizar la usabilidad en dispositivos con resoluciones bajas (evitando errores de Layout Overflow).
- **Control de Navegación:** Optimización del backstack (pila de actividades) para asegurar un flujo lógico de salida y retorno entre pantallas. Uso de finish() y el control del backstack evita que el usuario vuelva a pantallas de carga o login una vez dentro.
- **Seguridad Maestra:** Implementación de restricciones lógicas para proteger cuentas raíz (patrón @admin) frente a modificaciones accidentales por parte de otros administradores.
- **Completado al 100%.** Se garantiza la adaptabilidad de la UI en resoluciones desde 5" a 6.7" mediante ScrollView. Las pruebas de caja negra confirman la inhabilitación inmediata y la reducción visual de opacidad en los slots de tiempo pasados, respondiendo de forma exacta a los criterios anti-retroactividad horaria del negocio.

2. Seguimiento de Tareas Técnicas

ID	Tarea	Estado	Observaciones Técnicas
#49 T5.1	Mitigación de Inconsistencias	100%	Refactorización de la clave reservedSlots al campo unificado slots.
#50 T5.2	Rediseño de Interfaces Críticas	100%	Integración de Material Design 3, ScrollView y opacidad dinámica.
#51 T5.3	Refinamiento de Metadatos y Reglas	100%	Validación perimetral de campos de control (activo, rol, isAdmin).
#54 T5.4	Centralización de Feedback y Conectividad	100%	Implementación de isNetworkAvailable() y control preventivo de red.

3. Gestión de Incidencias de Calidad Finales

#50 Incidencia 22: Bloqueo de Interfaz por "Layout Overflow"

- **Tarea Relacionada:** T5.2 (Consistencia Visual)
- **Descripción:** Se detectó que en dispositivos con pantallas pequeñas o con el zoom de fuente activado, el botón de "Registrar" quedaba fuera del área visible y no era accesible.
- **Solución:** Sustitución del contenedor raíz por un **ScrollView**, permitiendo el desplazamiento vertical y garantizando que la App sea funcional en el 100% del parque de dispositivos Android.

#50 Incidencia 23: Fugas en el Backstack de Navegación

- **Tarea Relacionada:** T5.2 (lógica de navegación)
- **Descripción:** Al navegar del Registro al Login, el sistema mantenía la actividad anterior en memoria, permitiendo retrocesos innecesarios que confundían al usuario.
- **Solución:** Optimización de transiciones críticas mediante el uso sistemático de **finish()**, asegurando que el flujo de pantallas sea unidireccional y limpio.

#51 Incidencia 24: Vulnerabilidad en la Jerarquía de Administradores

- **Tarea Relacionada:** T5.3 (Refinamiento de Metadatos de Registro y Validación de Reglas).
- **Descripción:** Riesgo de que un administrador secundario pudiera editar o eliminar los permisos de la cuenta principal del centro.

- **Solución:** Implementación de un **bloqueo por código (hardcoded check)** basado en el patrón @admin. El sistema identifica la cuenta raíz y deshabilita los controles de edición de rol para ese UID específico, garantizando la persistencia de la cuenta maestra.

#51 Incidencia 25: Errores de Autenticación por Espacios en Blanco

- **Tarea Relacionada: T5.3** (Refinamiento de Metadatos y Reglas).
- **Descripción:** Algunos usuarios reportaban fallos al loguearse a pesar de poner la contraseña correcta. Se detectó que el teclado de Android a veces añade un espacio al final del email.
- **Solución:** Implementación sistemática de la función **.trim()** en todos los campos de entrada de Login y Registro. Con esto se asegura que los espacios accidentales no corrompan la cadena de texto enviada a Firebase Auth.

#51 Incidencia 26: Unificación del Protocolo de Seguridad en el Acceso (Login)

- **Tarea Relacionada: T5.3** (Refinamiento de Metadatos y Reglas).
- **Descripción:** Se detectó que el flujo de "Cambio Obligatorio de Contraseña" solo se aplicaba al rol de Cliente, dejando una brecha de seguridad en las cuentas de Empleados y Administradores tras un reseteo de credenciales.
- **Solución:** Refactorización del método checkUserStatusAndRole para implementar una **navegación dinámica mediante parámetros genéricos**. Ahora, todos los roles pasan obligatoriamente por el filtro de seguridad antes de acceder a sus respectivos paneles, garantizando que ninguna cuenta (especialmente las de gestión) opere con contraseñas temporales o débiles.

#51 Incidencia 27: Validación Robusta de Credenciales mediante Regex

- **Tarea Relacionada: T5.3** (Refinamiento de Metadatos y Reglas).
- **Descripción:** El sistema permitía contraseñas de baja complejidad (solo 6 caracteres), vulnerando la integridad de los datos de la peluquería ante ataques de fuerza bruta.
- **Solución:** Implementación de un motor de validación basado en **Expresiones Regulares (Regex)** en el cliente. Se fuerza un patrón de seguridad complejo: mínimo 8 caracteres, presencia de mayúsculas, números y caracteres especiales (@#\$%^&+=!), bloqueando el registro de cuentas vulnerables.

#54 Incidencia 28: Latencia Visual en la Carga de Slots

- **Tarea Relacionada: T5.4** (Centralización de Feedback y Control Preventivo de Conectividad)
- **Descripción:** Al cambiar de fecha, la interfaz mostraba slots antiguos durante unos milisegundos mientras esperaba la respuesta de Firestore, o no bloqueaba las horas pasadas inmediatamente.
- **Problemas Generados:** Sensación de lentitud y riesgo de que el usuario pulse una hora caducada antes de que el servidor responda.

- **Solución:** Optimización del ciclo de vida del método fetchOccupiedSlots. Se ha implementado un refresco preventivo de la UI **antes** de lanzar la petición asíncrona a la base de datos. Esto elimina el retardo visual y mejora la fluidez de la agenda.

#54 Incidencia 29: Ausencia de Feedback en Modo Offline

- **Tarea Relacionada:** T5.4 (Centralización de Feedback y Control Preventivo de Conectividad)
- **Descripción:** Al intentar realizar un login sin conexión a internet, la aplicación no emitía ningún aviso. Debido a la persistencia local de Firestore, la tarea quedaba pendiente en segundo plano sin informar al usuario.
- **Problemas Generados:** Incertidumbre en el usuario; el cliente no sabe si la reserva se ha procesado correctamente o si ha habido un fallo.
- **Solución:** Implementación de un ConnectivityManager para detectar el estado de la red antes de iniciar transacciones críticas, lanzando un aviso visual (Toast/Snackbar) informativo sobre el estado offline.

#54 Incidencia 30: Silencio Visual y Fallo de Anclaje en Mensajería

- **Tarea Relacionada:** T5.4 (Centralización de Feedback y Control Preventivo de Conectividad)
- **Descripción:** Se detectó que en estados de pérdida de conexión, el sistema de avisos mediante Snackbar fallaba silenciosamente al intentar anclarse a una vista raíz que no siempre estaba disponible o enfocada.
- **Problemas Generados:** El usuario pulsaba el botón de Login sin internet y la App no reaccionaba, dando la sensación de que el software estaba bloqueado ("congelado").
- **Solución:** Refactorización del método showMsgCustom para incluir un sistema de **anclaje dinámico por referencia de Vista (View v)** y un mecanismo de **"Fallback" mediante Toast**. Si el componente visual complejo falla, el sistema lanza automáticamente un aviso de sistema que garantiza que el usuario siempre reciba el feedback de "Sin conexión".

#54 Incidencia 31: Latencia en la Detección de Hardware de Red

- **Tarea Relacionada:** T5.4 (Centralización de Feedback y Control Preventivo de Conectividad)
- **Descripción:** El ConnectivityManager de Android presentaba un ligero retardo (latencia) en actualizar el estado de la red al cambiar a Modo Avión en el emulador/dispositivo.
- **Problemas Generados:** Posibilidad de lanzar una petición a Firebase Auth que quedaría en bucle de reintento infinito, consumiendo recursos.
- **Solución:** Implementación de una comprobación booleana estricta isNetworkAvailable() antes de cualquier llamada al SDK de Firebase. Se integró esta lógica en el botón de acceso para "frenar" la ejecución de forma inmediata si

no hay una red activa confirmada, mejorando la respuesta de la App de varios segundos a milisegundos.

#54 Incidencia 32: Fallo de Mapeo Estructural en Duración de Citas

- **Tarea Relacionada:** T5.4 (Centralización de Feedback y Control Preventivo de Conectividad)
- **Descripción:** Se detectó que al listar el historial de reservas en el cliente, todas las citas mostraban de manera invariable una duración fija de "30m", independientemente de si el servicio original contratado requería 45, 60 o más minutos.
- **Problemas Generados:** Mala experiencia de usuario (UX) e incertidumbre para el cliente, quien visualizaba en su pantalla un intervalo de tiempo inferior al que realmente había reservado en el centro de estética.
- **Solución:** Refactorización de la función de lectura asíncrona dentro del método. Se corrigió el error de tipado sustituyendo la clave errónea por el identificador de campo correcto indexado en la base de datos de Firestore. Con esto, el bucle procesa con éxito la longitud real del array en cascada y calcula la duración matemática exacta.

#54 Incidencia 33: Latencia por Intentos Asíncronos de Red en Reserva de Citas

- **Tarea Relacionada:** T5.4 (Centralización de Feedback y Control Preventivo de Conectividad)
- **Descripción:** Si un usuario intenta confirmar una reserva en un microcorte o sin internet, el SDK de Firebase Auth o Firestore entraría en un bucle infinito de reintentos en segundo plano. Esto provocaría latencia ("lagueo") visual e incertidumbre en el cliente al no recibir respuesta inmediata.
- **Solución Técnica Aplicada:** Se introduce una comprobación booleana estricta (mediante ConnectivityManager y NetworkCapabilities) integrada justo antes de ejecutar métodos transaccionales concurrentes (como db.runTransaction) en ReservarCitaActivity.
- **Impacto en el desarrollo:** Al "frenar" la ejecución en el hilo principal antes de invocar los listeners asíncronos de Firebase, el tiempo de respuesta del fallo de red en la interfaz pasa de varios segundos a milisegundos directos en el dispositivo local, desplegando un Snackbar rojo y amarillo instantáneo.

3. Comentario Técnico Final del Proyecto

Con la finalización de los cinco Sprints, el sistema alcanza el estado de Completado. Se han validado el 100% de los flujos operativos, garantizando que:

1. La gestión de roles es íntegra y jerárquica.
2. La visualización de la agenda es coherente y reactiva.
3. La protección de datos y la persistencia NoSQL cumplen con los estándares de seguridad y rendimiento propuestos.